

HW2

Quine-McCluskey Algorithm

2022202109

이호정

A. Problem state

Quine-McCluskey algorithm 란?

Quine-McCluskey algorithm 은 주어진 논리식을 최소화 하는 방법이다. 기존에 있는 카르노맵과 달리 *Quine-McCluskey algorithm* 는 변수가 많더라도 효율적으로 논리 최적화할 수 있다. *Quine-McCluskey algorithm* 의 실행시간은 입력에 대해 지수적으로 증가한다. 따라서 알고리즘을 짤 때 시간 복잡도를 계산하고, 코드 실행시간이 오래 걸리지 않도록 짜는 것을 고민했다.

Quine-McCluskey Method 방법

QM 알고리즘으로 문제를 풀 때는 다음과 같이 단계를 나눌 수 있다. Minterm 과 don't care 항의 숫자(이진수)에 존재하는 1의 개수를 기준으로 모든 항들을 그룹핑 한다. 1이 0개인 그룹, 1이 1개인 그룹... 그리고 1이 max_bit 개 인 경우를 마지막으로 그룹핑을 완료한다. 그렇게 1의 개수로 그룹을 나눈상태에서 이제 Prime implicant를 찾아내는 작업을 한다. 각각 옆에 있는 그룹에서 숫자를 비교할 것이다. 왜 바로 옆 그룹에서 찾아내는가 하면, 그룹이 하나 이상 차이 나게 되면 이미 hamming distance 차이가 1 이상 이 나버리게 된다. 그렇게 때문에 1의 개수가 하나씩 만 차이나는 그룹끼리 비교해주는 것이다. 그리고 같은 그룹에 있는 숫자들 끼리는 hamming distance 차이가 1이 날 수가 없기 때문에 같은 그룹 끼리는 비교해 줄 필요가 없다. 그렇게 숫자들을 비교해주는데, 비교할때는 숫자가 가진 1을 기준으로 비교해주면 쉽다. 숫자 1001 과 1011 이 있다고 가정하자. 이럴땐 10-1 이런식으로 다른 자릿수를 '-' 로 묶어주며 prime implicant 를 찾아주는 것이다. 이렇게 한 차례 검사를 마치면 column1 이 완성되게 된다. 그리고 완성된 coulumn1 을 기준으로 또 다시 위와 같은 과정을 반복한다. 이번에는 1 이 아니라 '-' 을 기준으로 묶어주면 된다. 이렇게 다른 숫자들과 묶이면 체크 표시를 해준다. 이런 과정에서 어떤 숫자와도 묶이지 않는 숫자들은 별표를 표기해 마지막 step으로 넘겨준다. 마지막 step 에서는 위의 step에서 묶어준 PI 와 minterm들을 아래표와 적는다.

True minterms		4	5	6	9	10
		0100	0101	0110	1001	1010
Prime Implicants	1010					●
	0-00	●				
	1-01				●	
	01--	●	●	●		
	-1-1		●			

위쪽에는 minterm, 왼쪽에는 속아낸 PI들을 적고 minterm 들 속하는 pi들을 체크해 주고 마지막 최소화를 진행한다. 먼저 minterm 1010에 대응 되는 PI 1010 은 유일하기에 최종 PI로 뽑아준다. 이런식으로 최종화를 진행하면 PI -> 1010, 1-01, 01 - - 이렇게 나오는 것을 알 수 있다. 이렇게 간소화를 진행하는 것이 QM algorithm 의 전반적인 과정이다.

B. Persu Code & Flow Chart

▶ Persu code

헤더파일 선언

FileHandler class 생성

-private

Minterm을 저장할 vector , minterm의 수 를 저장할 정수형 변수

-public

readMinterm 함수, writeResult 함수, getNum 함수 존재

▷ readMinterms 함수

파일열기 ⇒ 첫 번째 줄 읽기 (비트 길이) ⇒ minterm 들 읽기 ('m'으로 시작하는 줄만 저장) ⇒ 파일 닫기 ⇒ 반환값 _minterm 목록

▷ writeResult 함수

파일열기 ⇒ 결과값을 파일에 입력 ⇒ 파일 닫기

▷ getNum 함수

비트수를 저장후 반환

MintermProcessor class 생성

-private

Minterms 에 원본 minterm_in 을 저장

*-public : singleBitDifference 함수, generatePrimeImplicants 함수, covers 함수
minimizeFunction 함수, getMinimizedExpression 함수 가 존재*

▷ **singleBitDifference** 함수

두 숫자 간에 1 차이나는 것 찾기 ⇒ 찾으면 위치 반환, or 못찾으면 -1 반환

▷ **generatePrimeImplicants** 함수

Minterm 들을 그룹핑 ⇒ 바로 옆 그룹의 minterm 들을 비교하여 한 비트만 차이나는 애들끼리 묶어 새로운 minterm을 만듦 ⇒ 더 이상 minterm 생성이 불가능할 때까지 반복
⇒ 그후에도 남은 minterm 들을 implicant로 저장

▷ **covers** 함수

마지막으로 implicant 가 minterm을 cover 하고 있는지를 검사 ⇒ true/false를 반환

▷ **minimizeFunction** 함수

Cover 되는 implicant 목록 **생성** ⇒ prime implicant들을 찾아 최소화된 표현을 목록에 추가한다.

▷ **getMinimizedExpression** 함수

최종 최소화된 표현 출력

메인 함수

FileHandler instance 생성 ⇒ MintermProcessor instance 생성 ⇒

minterm 읽기 ⇒ prime implicant 생성 ⇒ 최소화 ⇒ 최소화된 표현 출력 및 파일 쓰기

Persudo Code _ 작성한 코드를 쉽게 간략화 해보았다. 코드를 보기 쉽게 바꾸어 작성했다.

(그냥 코드는x)

```
// FileHandler 클래스 정의
class FileHandler {
private:
    std::vector<std::string> minterms; // minterm들을 저장하는 벡터
    int num; // minterm의 수

public:
    // 파일로부터 minterm 읽기
    void readMinterms(const std::string& filename) {
        std::ifstream file(filename);
        if (!file.is_open()) {
            throw std::runtime_error("Unable to open file");
        }

        std::string line;
        getline(file, line);
        num = std::stoi(line);

        while (getline(file, line)) {
            if (line[0] == 'm') {
                minterms.push_back(line.substr(2));
            }
        }
        file.close();
    }

    // 결과값 txt에 작성
    void writeResult(const std::string& filename, const std::vector<std::string>& results) {
        std::ofstream file(filename);
        if (!file.is_open()) {
            throw std::runtime_error("Unable to write to file");
        }

        for (const auto& result : results) {
            file << result << std::endl;
        }
        file.close();
    }

    // Getter 함수
    const std::vector<std::string>& getMinterms() const { return minterms; }
    int getNum() const { return num; }
};
```

```

// MintermProcessor 클래스 정의
class MintermProcessor {
private:
    std::vector<std::string> minterms; // 원본 minterm들 저장
    std::vector<std::string> primeImplicants; // prime implicant 저장
    std::map<int, std::set<std::string>> groups; // 그룹을 bit수에 따라 저장

public:
    // 생성자
    MintermProcessor(const std::vector<std::string>& minterms_in) : minterms(minterms_in) {}

    // 두 minterm 사이에 단 하나의 bit만 차이나는지 확인
    int singleBitDifference(const std::string& a, const std::string& b) {
        int count = 0, pos = -1;
        for (int i = 0; i < a.size(); ++i) {
            if (a[i] != b[i]) {
                count++;
                pos = i;
                if (count > 1) return -1; // 1개 이상의 차이는 안되게함
            }
        }
        return count == 1 ? pos : -1; // 정확히 1개의 차이만 있어야 함
    }

    // prime implicant 생성하는 로직
    void generatePrimeImplicants() {
        // 초기 그룹화: 비트 수에 따라 minterm들을 그룹화 하기
        for (const auto& minterm : minterms) {
            int count = std::count(minterm.begin(), minterm.end(), '1');
            groups[count].insert(minterm);
        }

        // implicant 비교 및 결합 하기
        bool moreToCombine = true;
        while (moreToCombine) {
            moreToCombine = false;
            std::map<int, std::set<std::string>> newGroups;

            for (auto it = groups.begin(); it != std::prev(groups.end()); ++it) {
                auto& currentGroup = it->second;
                auto& nextGroup = std::next(it)->second;

                for (const auto& term1 : currentGroup) {
                    for (const auto& term2 : nextGroup) {
                        int diffPos = singleBitDifference(term1, term2);
                        if (diffPos != -1) {
                            moreToCombine = true;
                            std::string combinedTerm = term1;
                            combinedTerm[diffPos] = '-'; // 표시로 사용
                            newGroups[it->first].insert(combinedTerm);
                        }
                    }
                }
            }

            if (moreToCombine) {
                groups = newGroups;
            }
        }

        // 마지막으로 남은 implicant를 prime implicant로 저장
        for (const auto& group : groups) {
            primeImplicants.insert(primeImplicants.end(), group.second.begin(), group.second.end());
        }
    }

    // 최소화된 표현 반환 하기
    const std::vector<std::string>& getMinimizedExpression() {
        return primeImplicants;
    }
};

```

```

int main() {
    try {
        // 파일 핸들러 instance 생성 하기
        FileHandler fileHandler;

        // minterm 읽기
        std::vector<std::string> minterms = fileHandler.readMinterms("input_minterms.txt");

        // minterm processor instance 생성 및 초기화
        MintermProcessor mintermProcessor(minterms);

        // prime implicant 생성
        mintermProcessor.generatePrimeImplicants();

        // 최소화된 불리언 표현 결과 얻기
        std::vector<std::string> minimizedExpression = mintermProcessor.getMinimizedExpression();

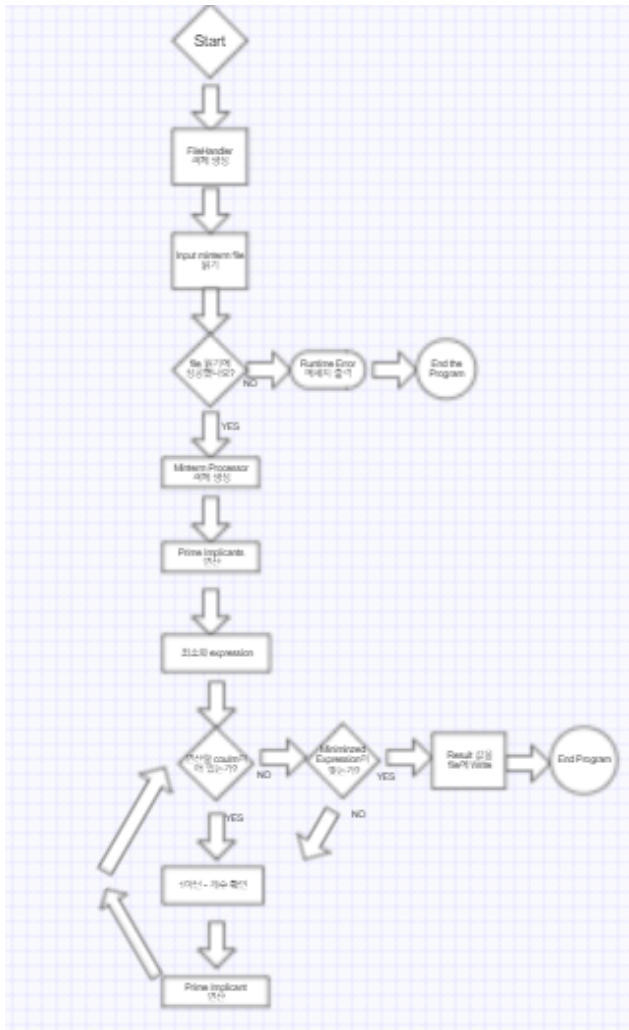
        // 최소화된 결과를 출력 파일에 기록
        fileHandler.writeResult("result.txt", minimizedExpression);

        // 결과 콘솔에 출력
        std::cout << "Minimized Expression:" << std::endl;
        for (const auto& expr : minimizedExpression) {
            std::cout << expr << std::endl;
        }
    } catch (const std::exception& e) {
        // 예외 처리: 오류 메시지 출력
        std::cerr << "An error occurred: " << e.what() << std::endl;
        return 1; // 오류 코드 반환
    }

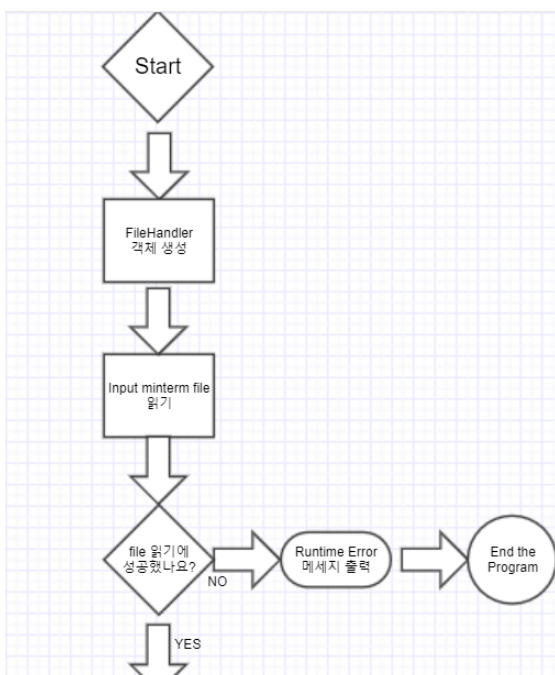
    return 0; // 성공적인 실행을 나타내는 0 반환
}

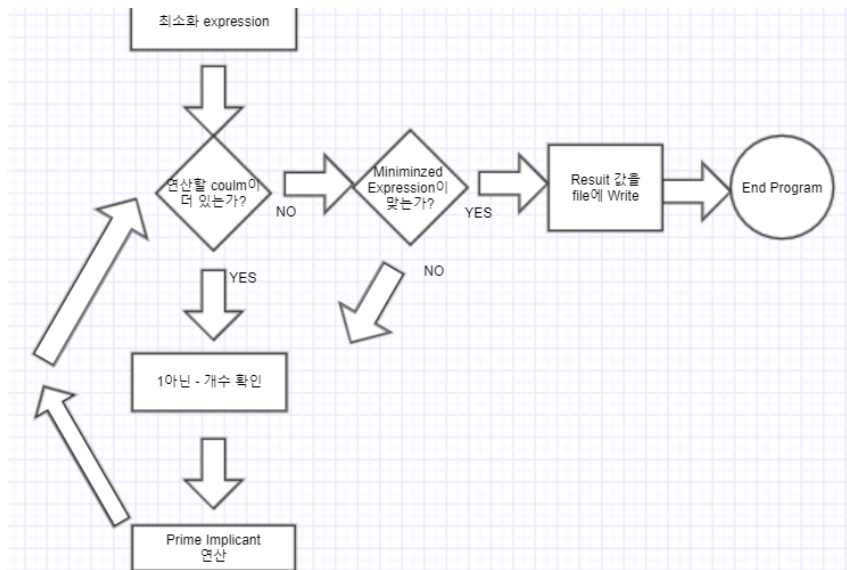
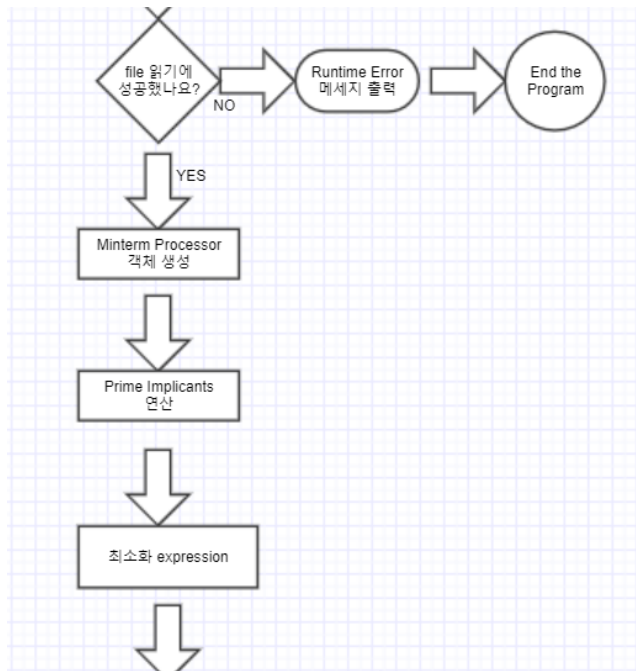
```

► Flow Chart



밑에 사진은 잘 보이지 않아서 확대해 삽입





C. Verification strategy & corresponding examples

► Strategy

작성한 코드에 대한 검증을 할 수 있는 전략으로는 여러 방법이 있다. 먼저 각각의 class 와 그 안에 속한 함수들이 제대로 동작하는지를 단계적으로 확인하고 메인 함수에서 까지의 흐름이 완벽하게 처리 되는지를 확인 하는것이다. 각각의 class 함수들만 확인 하는 것이 아니라 각각의 class 가 서로 간섭할 때 제대로 작동이 되는지도 확인해야 한다. 이렇게 코드를 나누어 단계적으로 밑에서부터 검증하는 방식도 있고, 전체 시스템을 테스트 하는 방법도 있다. 여기에는 손으로 계산 가능한 입력들을 넣어보고 결과가 똑같이 나오는지 확인 하는것이다. 이 방법은 큰 수 나 많은 입력들은 확인하기 어렵다는 단점이 있지만, 보다 직관적이기 때문에 빠르고 확실하게 검증 할 수 있다는 장점이 있다. 예를들면 손으로 계산할 수 있는 입력들을 입력 한다면, 아니면 최소화가 불가능한 예시들을 넣어보는것이다.

► Example 1

4

0000
0100
1000
1010
1011
1100
1101
1111

내 코드에 왼쪽 사진의 숫자들을 input 으로 넣었을 때 나오는 결과 값이 오른쪽이다. 이를 직접 풀어서 검증 해보자

--00

아래 직접 손으로 푼 QM 알고리즘과 결과 값이 똑같이 나오는걸 확인 할 수 있다. 이렇게 간단한 예제 말고 복잡한 예제도 확인해 볼 수 있다.

Example) 0, 4, 8, 10, 11, 12, 13, 15

0 → 0000 0100 1000 1010 1011 1100 1101 1111

			column 1	column 2
G ₀	0	0000 ✓	0-00 ✓	--00 ⊗
G ₁	4	0100 ✓	-000 ✓	--00 ⊗
	8	1000 ✓	-100 ✓	--00 ⊗
			10-0 ⊗	
G ₂	10	1010 ✓	1-00 ✓	
	12	1100 ✓	--00 ✓	
			101- ⊗	
G ₃	11	1011 ✓	110- ⊗	
	13	1101 ✓	1-11 ⊗	
			11-1 ⊗	
G ₄	15	1111 ✓		

	0000	0100	1000	1010	1100	1101	1111
10-0			✓				
101-				✓		✓	
110-					✓		
1-11							✓
11-1							
--00	⊙	✓	✓		✓		

► Example 2 : $f(A,B,C,D,E) = \sum m(1,3,7,8,10,15,16,19,21,23,27,30,31)$

5
 m 00001
 m 00011
 m 00111
 m 01000
 m 01010
 m 01111
 m 10000
 m 10011
 m 10101
 m 10111
 m 11011
 m 11110
 m 11111

► 내 코드에 넣은 input

--111
 000-1
 010-0
 1--11
 10000
 101-1
 1111-

► 메모장에 나온 결과값

먼저 각 숫자들을 이진수로 변환하고 1의 개수에 따라 그룹핑을 한다. 그리고 prime implicant 연산을 수행한다.

G ₁	00001 ✓	000-1 *
	01000 ✓	010-0 *
	10000 *	--0-11
G ₂	00011 ✓	-0011 ✓
	01010 ✓	00-11 ✓
		--111 *
G ₃	10011 ✓	10-11 ✓
	10101 ✓	101-1 *
	00111 ✓	0-111 ✓
G ₄	01111 ✓	-0111 ✓
	10111 ✓	-1111 ✓
	10011 ✓	1-111 ✓
	11110 ✓	11-11 ✓
G ₅	11111 ✓	1111- *

그룹핑하고 column을 두 번 구하면 위 와 같이 나온다 . 이때 implicant는 10000 ,000-1 ,010-0 , 101-1, 1111-, -0-11, 1--11, --111 이렇게 8개 가 나온다.

그리고 이 implicant 들을 minterm 과 표를 만들어 최종적으로 검사하면 다음과 같다.

	00001	01000	10000	00011	01010	10011	10101	00111	01111	10111	11011	11110	11111
10000			✓										
000-1	✓			✓									
010-0		✓			✓								
101-1							✓			✓			
1111-												✓	✓
-0-11				✓				✓		✓			
1--11						✓				✓	✓		✓
--111								✓	✓	✓			✓

	0000	0100	1000	0001	0101	1001	0111	0011	0111	1011	1101	1110	1111
1000			✓										
0001	✓			✓									
0100		✓			✓								
1011						✓				✓			
1111												✓	✓
0011				✓				✓		✓			
1011					✓					✓	✓		
1111								✓	✓	✓			✓

Prime Implicant: 1000, 0001, 0100, 1011, 1111, 0011, 1011, 1111

그리고 이 implicant 들이 minterm 들을 커버하는지 검사하고 최종 prime implicant 를 추출해 내면 위의 표와 같이 나온다.

최종 implicant는 1000, 0001, 0100, 1011, 1111, 0011, 1011, 1111 으로, 내 코드와 똑 같은 결과값이 나오는 것을 확인 할 수 있다.

D. TestCase

1. Testcase 1

▶ 100bit 짜리 input 1000개

```
input_minterm
× +
파일 편집 보기

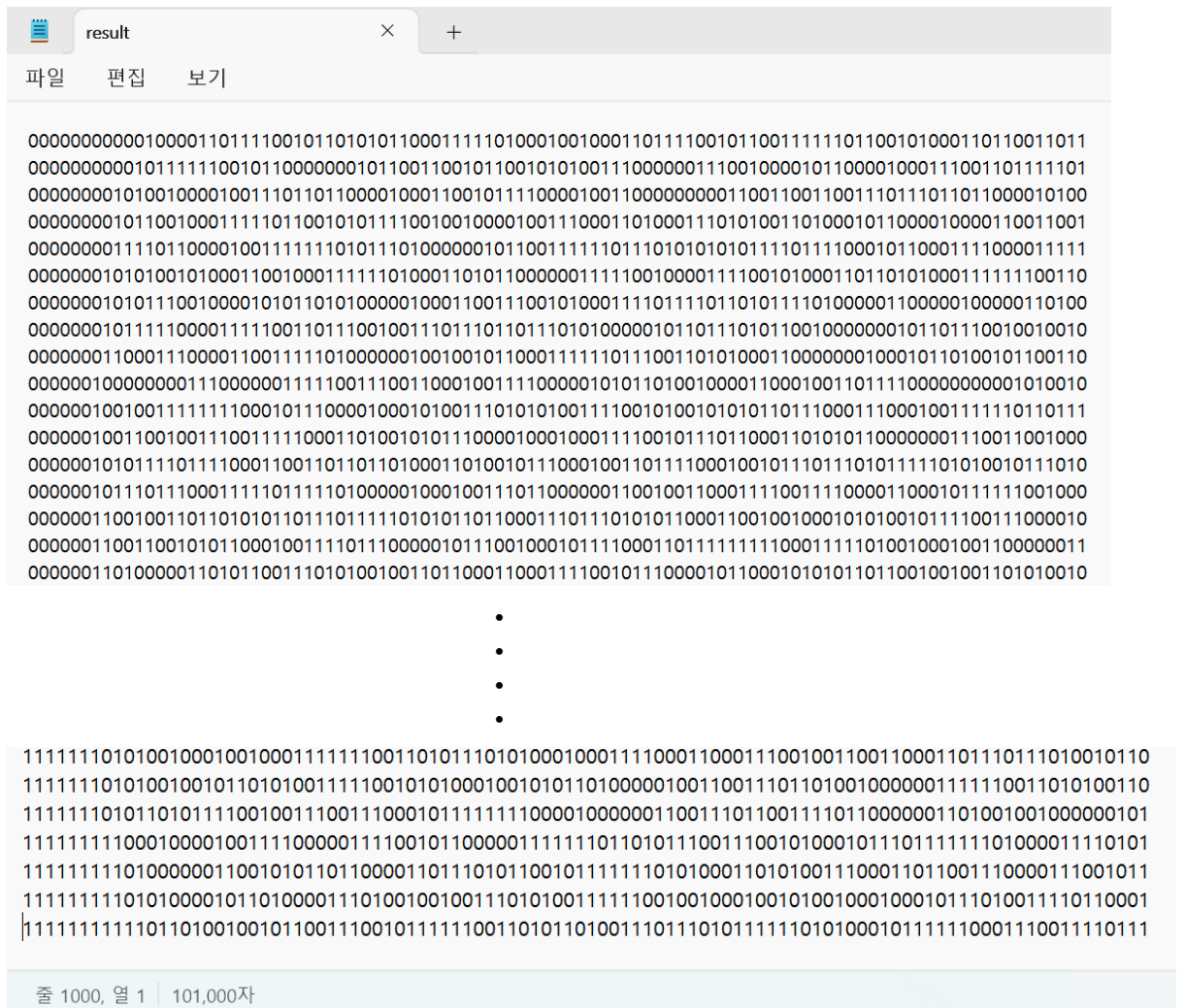
100
m 1001000011100001011011111110000101000001001100100010111010111100100001010100001101111010111100100
m 0011111010111100111001111110010110010010010101110001001111000011011110000001001101001111000111010
m 11111011110011110000000101100101110101111010000000001110011111000100010101100010001001100001001111
m 011110111110000100001011101101110100101011010001001001010110100110011101110111100110010000010100
m 01000011010000010001001011001010100000001010110111011101100111001100011100010001110011100111010001
m 0011000011000001111110111010100111000000110101010000101011101101001011011001111001000100100010011
m 00110010000101010101001000001111010011110101001111101111011000100000010000001101010100000010110
m 000010110010101011110111100111010100110101010101111101010100001111000010011101010110000110001110
m 0110001001000111010001001000101100111101100101111101010101010101110111100101010000111010011010
m 1110110101000100010001111010101110001111110010100110100111011001100000011101101001011101001000100
m 01000010011110010100100100110001110101010010101100000010111100011011000011001010100101011111011
m 01100101110001001110011001101000010000100010011000100011101100000111011000100011100001010101
m 01001101110011011011111010111101101110100010011010000010010011100000011000111110101101010100001
m 11100000010010010111010000010011010101010101000101101101100010001000111000001010101010010110101010
m 110000011000110110000011100101101001101101100111000111101100111110011001110011000111010001110
m 1000110001011110111101011110010011111111100000100001000011100010011110110010010000000101010001000
m 10011111010010100011100100001101001110110111010100111100011101010111001110000000001110100110011010
m 000111000110101111010100011010111110011011111101000011100100010000000101000111011010001100100111
m 11100110000111100101110100001000101110101110101001100100100001000001001000001100111110010100100110
m 101011100001011001111000110010100101110101010101110010110100010011110111011000111101100101
m 001001010101001101001001001011000101111100000101110100000010111010100110111010011100110110000101
m 0101111000011000100010110111011111000111011110000011001110001000010100111010010010000010110101001
m 001000011111010010010101100001100000010011110001010110110011000010110100101100000101111110010010001
m 10100101110111001000100010001111011000010001011010010001110001000100001110010101001100111110010
m 0111010101000010010110001100100100110111000110110101100000010010001000000000100100010001010111001
m 0001110111100111000000000110000111000110011010110010011010110000110101010011101101110101101100
m 011101010001101011110111100111010101100011010000101110000010010010010010010001110100111101101000
m 11000100110111001111010000100011101110100011011101100001000001010101110111010100110000011010010
m 0000101000100100110100000000011110100000110110100011011111100101111011100111001010010001000010
m 111101100101101111010000110110000110111011100100100000100011100111011110000011111100001101101110
m 1101011101010111101001000100100010111101000000100001010011110000011011111000001011111010000011

.
.
.
.

m 01011110010111101000010101111011100111011010010101110011111010001110100011101000111011110000101100110011011
m 0100000010011011011100011101101011011101011001010001001100001001110100011110111100001011001100111011
m 001101100100011001100010011000010000111100111011111111010111011100100110000110110111111000111110
m 101110011110101001111111000110110000000111000101101100101111010110001101010100010001000100010011
m 01110011110011001000110011011111000100001101110010001110001111110001111000011110101100011101101010
m 0100001101101010111110111100010010000000100001101001010010001010111100001111010011100111111011
m 1010001011000110101100011000010101100010101100100001000010010110001111001100100011100001001010101
m 111111100010000111110100011101000000100110100111100011110010010110000101010001111011110001110000
m 011110101000011000000000111101010111101010000000001110001101101011100101000000001100010000000001001
m 110001111111000001111000101011001101101001101110101001001110010100111011101110101000100001011
m 0100001110000101110010110001100110111101110110010111111100111100000111111011010001110000101101011
m 0110011101111001011101000011000011111111110101010111001010100000101111000000111110100100011100
m 111101001111010111101110000110101001000101001110110010111101001111010011111110000001111101000011110
m 1011100110110110001101001101110010111011011010110000100000011101111010110000111111101011100100100
m 10001111111110001011101001011111000010010101101101010011000001010000110000111010000101101011100000
```

줄 1001, 열 1 | 103,003자

위와 같이 100bit 짜리 이진수 1000개를 출력한 후 값을 확인해 보았다.



```
0000000000010000110111100101101010110001111101000100100011011110010110011111101100101000110110011011
00000000000101111110010110000000101100110010110010100111000000111001000010110000100011100110111101
00000000010100100001001110110110000100011001011100001001100000000011001100110011101101101000010100
0000000001011001000111110110010101110010010000100111000110100011101010011010001011000010000110011001
0000000001110110000100111111010111010000001011001111101101010101110111100010110001111000011111
000000010101001010001100100011111010001101011000000111100100001110010100011011010100011111100110
00000001010111001000010101101010000010001100111001010001110111101101011101000001100000100000110100
00000001011111000011111001101110010011101110110101110100000110000010000011010010010010010
00000001100011100001100111110100000010010010110001111101110011010100011000000010001011001001100110
0000001000000001110000001111001110011000100111100000101011010010000110001001101110000000001010010
00000010010011111111000101110000100010100111010101001110010100101010110001110001001111110110111
00000010011001001110011111000110100101011100001000100011110010111011000110101100000001110011001000
0000001010111101110001100110110110100011010010111000100110111000100101110111010111101010010111010
00000010111011100011110111110100000100010011101100000011001001100011110011110000110001011111001000
00000010010011011010101101110111110101011011000111011101010100011001001000101010010111100111000010
00000010011001010110001001110111000001011001000101110001101111111100011111010010001001100000011
000000110100000110101100111010100100110110001100011110010111000010110010010011011001001101010010
```

...

```
1111111010100100010010001111111001101011101010001000111100011000111001001100110001101110111010010110
1111111010100100101101010011111001010100010010101101000001001100111011010010000001111110011010100110
11111110101010111100100111001110001011111110000100000011001110110011110110000001101001001000000101
111111111000100000100111100000111100101100000111111011010111001110010100010111011111101000011110101
11111111101000000110010101101100001101110101100101111111010100011010100111000110110000111001011
11111111101010000101101000011101001001001110101001111110010010001001001000100010111010011110110001
11111111111011010010010110011100101111100110101101001110111010111111010100010111111000111001110111
```

줄 1000, 열 1 | 101,000자

결과가 다음과 같이 나오는걸 확인할 수 있다. 아무래도 사이즈와 개수가 너무 많기 때문에 직접 계산해 볼 수는 없겠지만 output 개수가 input 과 같이 1000개 이기 때문에 프로그램이 제대로 돌아갔다는 것을 확인 할 수 있다. 경우의 수가 100비트 이진수로 2^{100} 이다 보니 겹쳐서 나올리가 없기 때문이다. 이렇게 큰 bit수의 input이 아주 많이 들어올 때 연산을 하는 것은 매우 복잡하다고 생각해서 예시 testcase 에 넣었다. 처음엔 그저 돌아가는 것만 생각했는데 이렇게 큰 수를 많은 양으로 다루게 된다면 시간복잡도와 효율성을 생각해야겠다는 깨달음을 얻고 코드를 일부 수정했다. 오히려 sort 를 시키는게

-Testcase2

위와 같이 크고 많은 input 을 넣었을 때 아무래도 bit 수가 크면 겹치는 수가 입력값이 없기 때문에 제대로 연산했는지 알아보기 어려울 수도 있다. 그래서 이번에는 비슷하게 100bit 2000개의 input을 생성하고 입력하는데 의도적으로 10개를 아예 똑같이 넣어서 입력해보겠다.

```
100
m 01010010111011001001001011101001100001111101110000000111011010101011111001010100101010010111100100
m 01010010111011001001001011101001100001111101110000000111011010101011111001010100101010010111100100
m 01010010111011001001001011101001100001111101110000000111011010101011111001010100101010010111100100
m 01010010111011001001001011101001100001111101110000000111011010101011111001010100101010010111100100
m 01010010111011001001001011101001100001111101110000000111011010101011111001010100101010010111100100
m 01010010111011001001001011101001100001111101110000000111011010101011111001010100101010010111100100
m 01010010111011001001001011101001100001111101110000000111011010101011111001010100101010010111100100
m 01010010111011001001001011101001100001111101110000000111011010101011111001010100101010010111100100
m 01010010111011001001001011101001100001111101110000000111011010101011111001010100101010010111100100
m 01010010111011001001001011101001100001111101110000000111011010101011111001010100101010010111100100
m 011010011000001010011010010001011111000101111110110011101101011110010011110111101111100100000110
m 1101100100100001110000110011001110101110010111111110011101101011110010011110111101111100100000110
m 0010011010111101100100110100000111110111001001101101000100110110110001100111110110010011111100010
m 0111100110000100010011101001010000100010001101010100110000110110100011001110000001101111010001111100
m 11101010100001010000110011100011010000100111011100111000111101100000011000110011100010010001011010
m 000001101000001010010100000101001101111010110010100101001001101110111111001010010100011000000011
m 001010101011111000100110100011111001000100001101001100011110001110110110101111011001100110111100
m 011001111001100011110011010111001100010000100110001101110110101001010011011001000110111101001111100
```

이렇게 10개의 input을 똑같이 넣어 보았다.

```
m 000110101101111101000010001011110011110100001110001010010011000010011111101000001011111110111011100
m 0111111110001100011101111010100000011011010111100001110010100110111010001011110111001011000010000100
```

줄 2001, 열 2 | 206,004자

2000개를 넣었다. 2001 개인 이유는 맨 윗줄 m 때문이다.

그리고 연산 결과를 보면 아래와 같다

```
111111101111111010111110111110100010001111011101000011111010001011000010010101001001101111101001
111111110011010001100000001100111111001001100110010000001011011000000110011111111001011110111101001
1111111100111110100011001110010110010001111011100010011000011110100010010011100100101100101111001101
111111110101101000011001001011101110010010011100001111110011110000001001010011000100001001000010111
1111111101100010011111101100100100100101110111000001010001101101011010010010010101111110011001010001
111111111010010111001010110010110001111111010001010111001110000010010100100001111011011000101111
11111111110011010010110001111111010001100010011000110101100110011100110000111010110110100101111
```

줄 1991, 열 5 | 201,091자

이렇게 1991 개로 줄어든 것을 볼 수 있다. 이 이유는 10개의 minterm 을 cover 하는 하나의 prime implicant로 줄여졌기 때문이다.